TOBB ETÜ

Ekonomi ve Teknoloji Üniversitesi

BIL 496 LOW-LEVEL DESIGN REPORT

ALİ BELİKIRIK

BEYZA DEĞİRMENCİ

İNAN ÖZŞAHİN

ÖZGE DOĞAN

Table Of Contexts

Table Of Contexts 2
1. Introduction 4
1.1 Object Design Trade-Offs 4
1.1.1 Modular Decoupling vs. Implementation Complexity 4
1.1.2 Data Normalization vs. Query Performance 4
1.1.3 Synchronous Processing vs. System Responsiveness 5
1.1.4 Type Safety vs. Generic Flexibility 5
1.2 Interface Documentation Guidelines 5
1.2.1 REST API Documentation Standards 5
1.2.2 Service Interface Contracts 5
1.2.3 Error Messaging and Localization 6
1.2.4 Versioning and Compatibility 6
1.3 Engineering Standards (e.g., UML and IEEE) 6
1.3.1 UML Modeling Standards 6
1.3.2 IEEE Software Documentation Compliance 6
1.3.3 Layered Architecture Standard 7
1.3.4 Security Standards 7
2. Packages 8
2.1 Model 8
2.2 Repository 8
2.3 Service 9
2.4 Controller 10
2.5 View 10
3. Class Interfaces 11
3.1 Model 11
3.1.1 ROLE Enum Class: 11
3.1.2 User Class: 11
3.1.3 DATA_TYPE Enum Class: 11
3.1.4 DataPacket Class: 11
3.1.5 SETTING_TYPE Enum Class: 12
3.1.6 PreproccesingSetting<T> Interface Class: 12
3.1.7 PreprocessingOperation Interface Class: 12
3.1.8 AiResult Class: 13
3.1.9 Dataset Class: 13
3.1.10 Preprocessor Class: 14
3.1.11 AiClient Class: 14
3.1.12 AUDIT_ACTION_TYPE Enum Class 14
3.1.13 AuditRecord Class 15
3.1.14 Report Class 15
3.2 Repository 16
Each repository shall be implemented as an interface and extend JpaRepository with the type of the class and Long (ex. public interface ObjectRepository extends

JpaRepository<ObjectModel ,Long>) JpaRepository will give abilities to access the database with its default implementation. 16
3.2.1 DataPacketRepository extends JpaRepository<DataPacket, Long>: 16
3.2.2 UserRepository extends JpaRepository<User, Long>: 16
3.2.3 DatasetPacketRepository extends JpaRepository<Dataset, Long>: 16
3.2.4 AuditRecordRepository extends JpaRepository<AuditRecord, Long>: 16
3.2.5 ReportRepository extends JpaRepository<AuditRecord, Long>: 16
3.3 Service 17
3.3.1 UserService Class 17
3.3.2 PreprocessService Class 17
3.3.3 AiModuleService Class 17
3.3.4 DataSetService Class 18
3.3.5 AuditService Class 18
3.3.6 ReportService Class 18
3.4 Controller 18
3.4.1 UserController Class , endpoint: ‘api/user’ 19
3.4.2 PreprocessController Class endpoint: ‘api/preprocess’ 19
3.4.3 AiModuleController Class endpoint: ‘api/aimodule’ 19
3.4.4 DataSetController Class endpoint: ‘api/dataset’ 19
3.4.5 AuditController Class endpoint: ‘api/audit’ 20
3.4.6 ReportController Class endpoint: ‘api/report’ 20
3.5 View 20
3.5.1 LoginView 20
3.5.2 MainView 21
3.5.3 DataSelectionView 21
3.5.4 PreprocessSettingView 21
3.5.5 AiSelectionView 22
3.5.6 ReportView 22
3.5.7 AdminPanelView 22
4. Glossary **23**
5. References **23**

1. Introduction

Artificial intelligence-powered solutions are increasingly prevalent in healthcare. However, data from devices may be inconsistent and incomplete for AI models. Furthermore, fully automated systems without expert physician interpretation may lead to incorrect decisions in patient treatment. Finally, the preprocessing required for data to train accurate AI models remains a research topic for engineers.

The NeoAI project aims to simplify collaboration between specialist doctors and engineers. It allows users to crop ultrasound image data and apply other defined preprocessing operations on demand and in the order requested. It then feeds the processed data into AI models and stores the results in a database along with the data itself. It also stores information about the history of operations performed on the data and who performed which operations, simplifying future AI research and simplifying control.

This report defines the low-level architecture of the preprocessing layer of the NeoAI project. Report comprises Object design trade-offs, Engineering standards, Packages and Class interfaces sections. Finally, the report is concluded with the class diagrams and the detailed explanations of software components.

1.1 Object Design Trade-Offs

In the architectural design of the AI-Enhanced Data Ingestion and Preprocessing Platform, several engineering trade-offs were evaluated to balance system performance:

1.1.1 Modular Decoupling vs. Implementation Complexity

The system follows a strict Service-Oriented Architecture where controllers communicate exclusively through service interfaces rather than direct implementation.

- **Trade-off:** Decoupling the business logic into separate layers increases the initial development time and introduces more classes and interfaces to manage.
- **Decision:** We prioritized modularity of the system. This ensures that the AI Module (FR4) or the Data Ingestion logic (FR1) can be updated or replaced independently without affecting the entire system. This flexibility is vital for future-proofing the platform as NICU medical standards evolve.

1.1.2 Data Normalization vs. Query Performance

The database schema utilizes a highly normalized structure, linking Audit Records, Users, and Datasets via Long-typed Primary Keys.

- **Trade-off:** High normalization prevents data redundancy and ensures integrity but requires multiple "JOIN" operations, which can slightly increase query latency compared to a flat, denormalized structure
- **Decision:** We prioritized data integrity and traceability (FR10). In a medical context, ensuring that every audit log is accurately linked to a specific user and data packet is more critical than raw microsecond performance. The use of indexed Long IDs mitigates the performance overhead of relational joins.

1.1.3 Synchronous Processing vs. System Responsiveness

The platform must handle high-frequency pulse oximeter data (FR1) while simultaneously generating audit logs (FR10) and AI inferences (FR4).

- **Trade-off:** Processing every step—such as saving an audit record or normalizing a signal—synchronously would block the user interface, leading to a poor experience for the clinician (FR2).
- **Decision:** We implemented an asynchronous logging and notification strategy. While critical data transformations are processed immediately to ensure accuracy, non-critical tasks like audit storage and dataset consolidation are handled in background threads. This ensures the dashboard remains responsive for real-time monitoring.

1.1.4 Type Safety vs. Generic Flexibility

The use of the PreprocessingSetting<T> interface and the Factory Pattern was a deliberate choice to handle different setting types (Integer, Float, String).

- **Trade-off:** Using generic types with a factory adds an abstraction layer that makes the code more verbose compared to using a simple "Object" type or a "String" for all settings.
- **Decision:** We prioritized Type Safety. By enforcing specific types at the design level, we prevent runtime errors during the preprocessing phase, ensuring that a "filter kernel size" can never accidentally be assigned a non-integer value.

1.2 Interface Documentation Guidelines

To ensure consistency and ease of integration across the platform's subsystems, all internal and external interfaces follow a standardized documentation protocol.

1.2.1 REST API Documentation Standards

All controllers are documented using the OpenAPI (Swagger) Specification. This includes:

- **Endpoint Definition:** Clear naming conventions following RESTful principles
- **Request/Response Payloads:** Detailed JSON schemas for every ResponseEntity
- **HTTP Status Codes:** Explicit documentation of success (2xx) and error (4xx, 5xx) codes to guide the Frontend implementation.

1.2.2 Service Interface Contracts

Since controllers communicate via Service Interfaces, each interface must include:

- **Method Signatures:** Clear definitions of generic types and their constraints.
- **Pre-conditions & Post-conditions:** Documentation of what the system state must be before calling the method and the guaranteed state after execution.
- **Side Effects:** Explicitly stating if a method triggers an AuditRecord (FR10) or modifies a DataPacket.

1.2.3 Error Messaging and Localization

Interfaces must follow a unified error handling guideline:

- **Standardized Error Objects:** Every failed request returns a consistent object containing an errorCode, a message, and the transactionId from the watermark.
- **Validation Rules:** Documentation of input constraints (e.g., "Kernel size must be an odd integer between 1 and 15") to prevent malformed data from reaching the Analytics Subsystem.

1.2.4 Versioning and Compatibility

- Deprecation Policy: Guidelines for marking interfaces as "Deprecated" before removal to ensure the MainController transitions smoothly to new service versions.

1.3 Engineering Standards (e.g., UML and IEEE)

The NeoAI platform follows internationally accepted software engineering principles to ensure maintainability, scalability, and regulatory compliance.

1.3.1 UML Modeling Standards

The system design is represented using UML artifacts:

- Class Diagrams to illustrate relationships between Model, Repository, Service, and Controller layers.
- Sequence Diagrams to represent the interaction flow between Frontend, Backend, Database, and AI Module.
- Layered Architecture Diagrams to demonstrate separation of concerns.

All diagrams follow standard UML 2.x notation.

1.3.2 IEEE Software Documentation Compliance

This Low-Level Design document is structured according to IEEE 1016 – Software Design Description guidelines. The document contains:

- Architectural decisions and trade-offs
- Package structure definitions
- Detailed class interfaces
- Persistence layer definitions
- View layer mapping

This ensures traceability between Functional Requirements (FR1–FR10) and implementation components.

1.3.3 Layered Architecture Standard

The platform follows a strict layered architecture:

1. Model Layer – Domain entities and enums classes
2. Repository Layer – Database abstraction via JPA
3. Service Layer – Business logic
4. Controller Layer – REST endpoints
5. View Layer – User interface

No layer directly accesses a lower layer except through defined interfaces. Controllers never access repositories directly; they communicate through service interfaces.

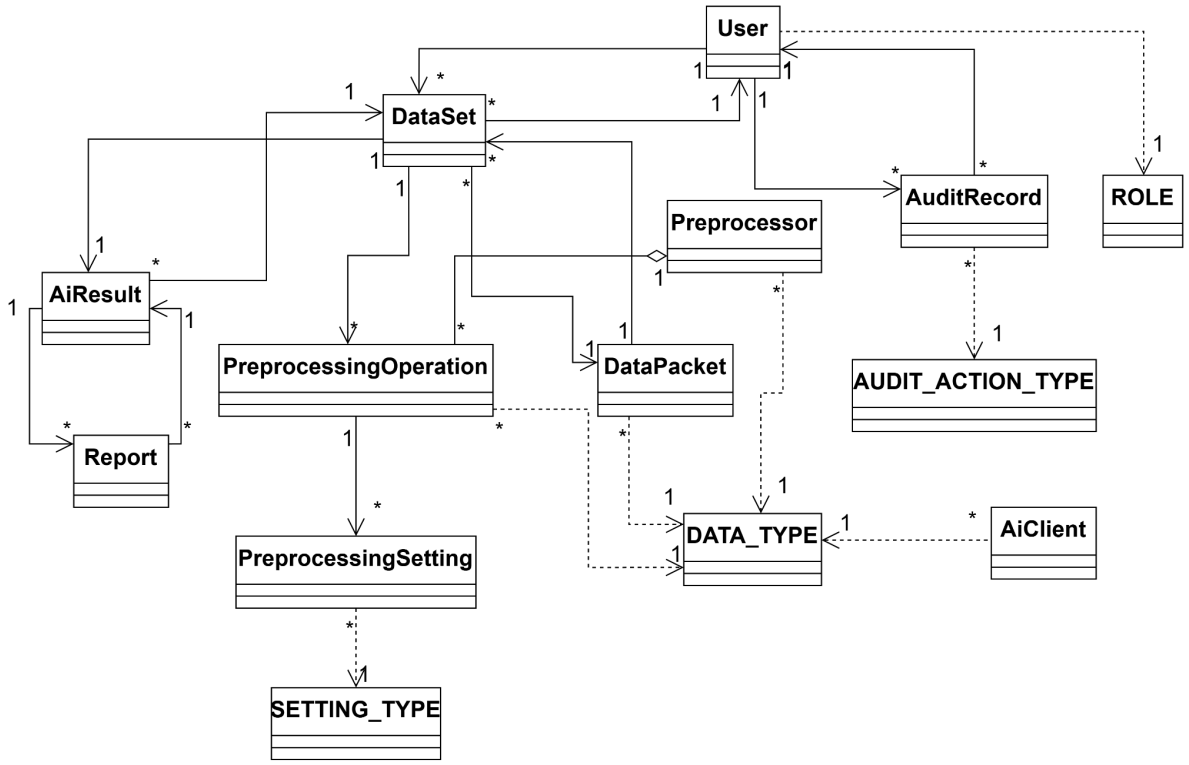
1.3.4 Security Standards

- Passwords must be stored as hashed values.
- All endpoints enforce role-based access control.
- Audit records are immutable.
- All critical actions generate an audit record.2. Packages

The system is organized into logical packages to ensure separation of responsibilities:

2. Packages

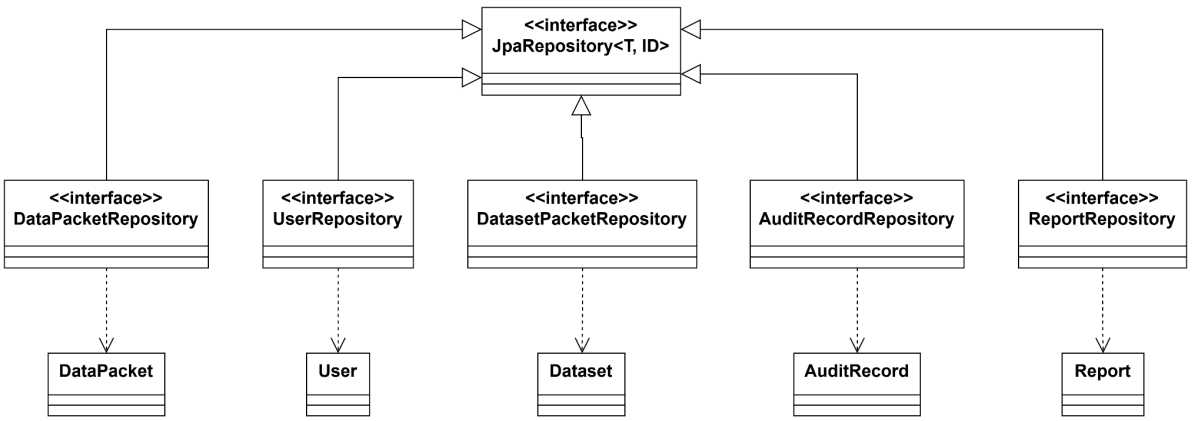
2.1 Model



Contains:

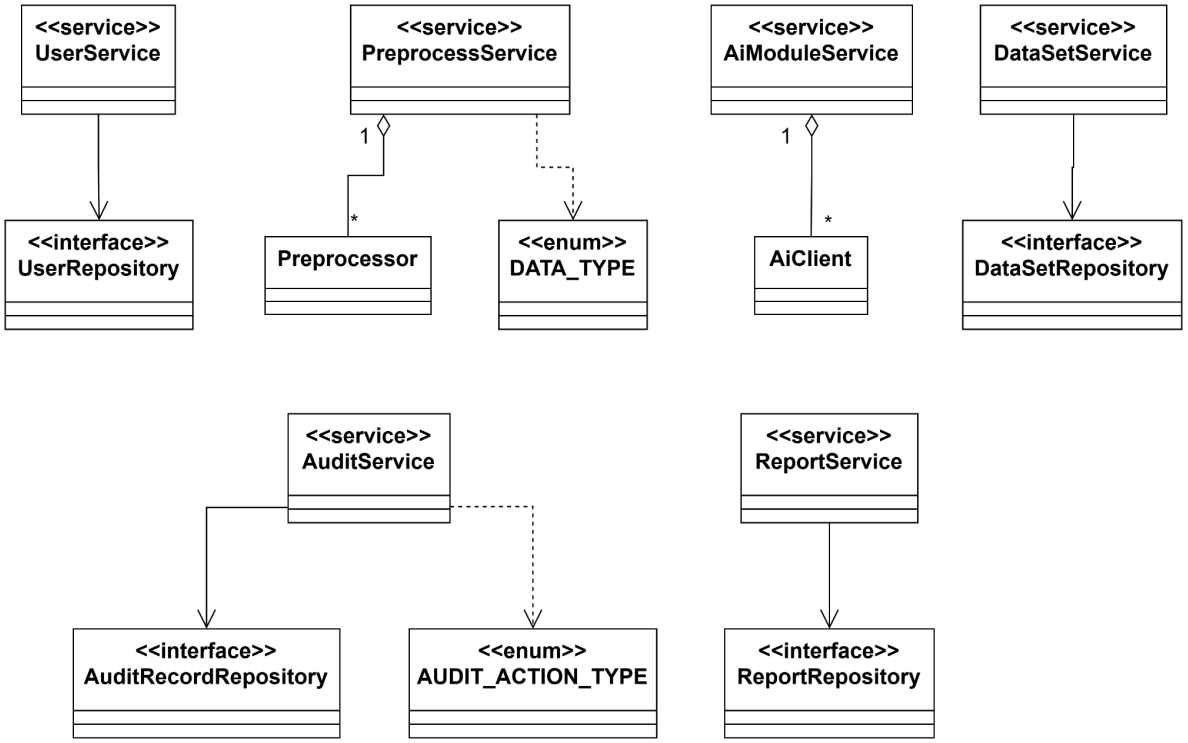
- Domain entities (User, Dataset, DataPacket, AuditRecord, AiResult, Report)
- Enums (ROLE, DATA_TYPE, SETTING_TYPE, AUDIT_ACTION_TYPE)

2.2 Repository



Contains JPA repository interfaces extending JpaRepository.

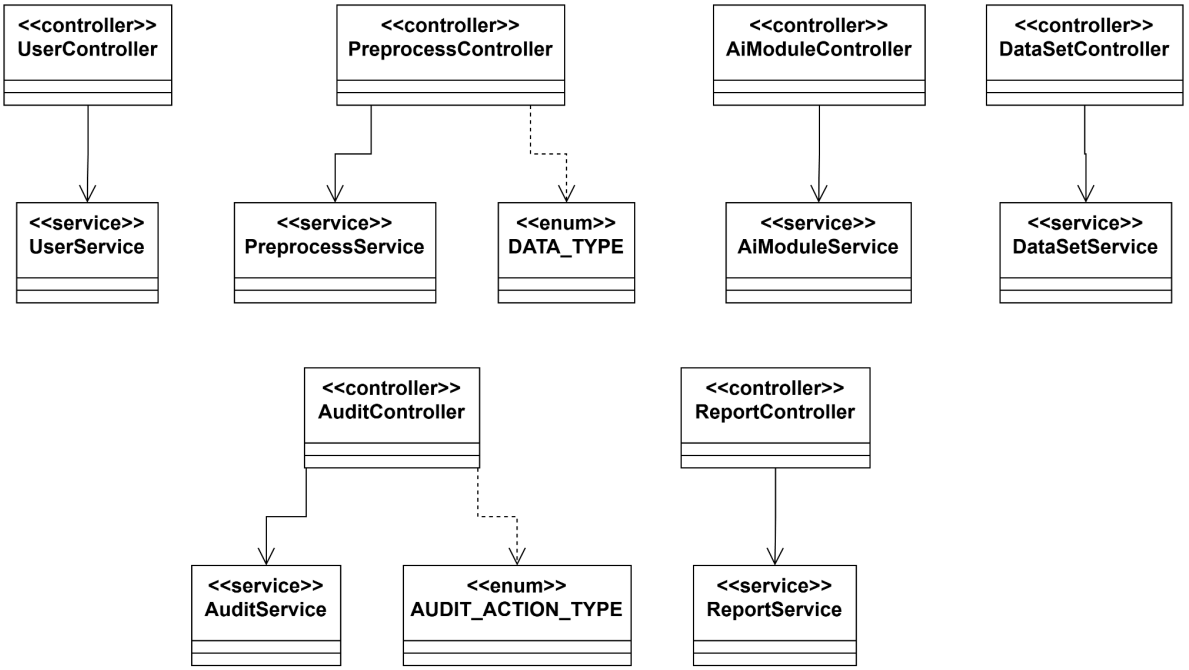
2.3 Service



Contains business logic classes:

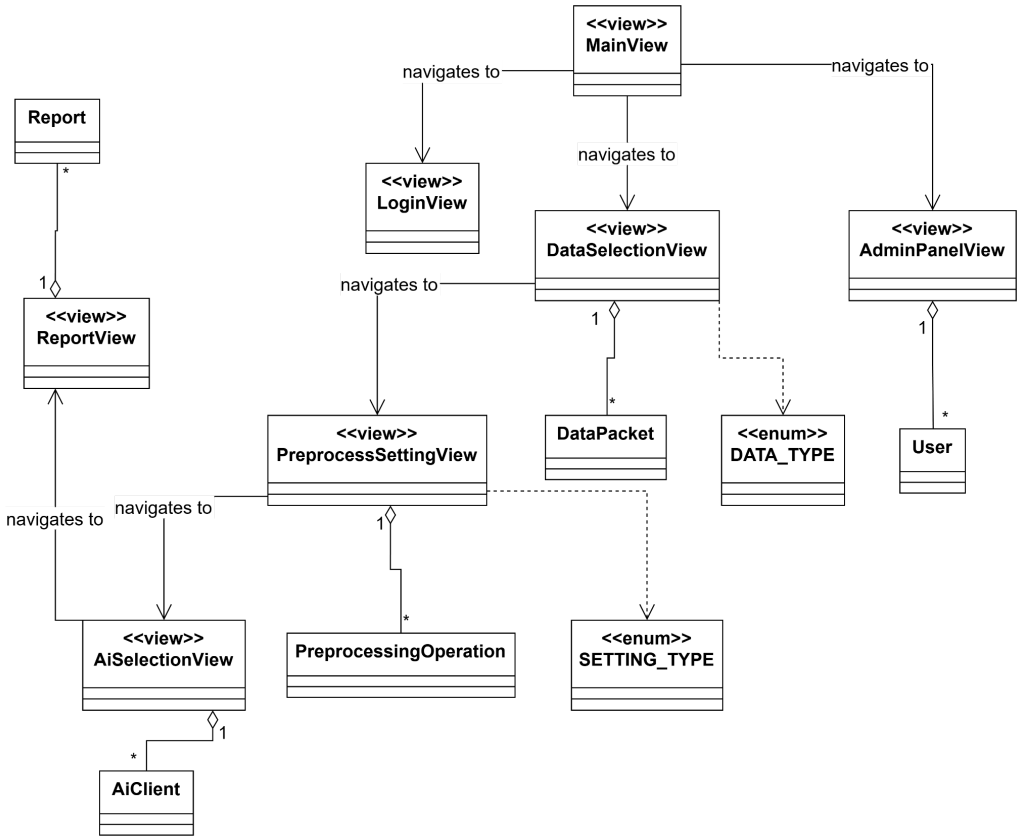
- UserService
- PreprocessService
- AiModuleService
- DataSetService
- AuditService
- ReportService

2.4 Controller



Contains REST controllers that expose endpoints.

2.5 View



Contains UI interaction classes.

3. Class Interfaces

3.1 Model

3.1.1 ROLE Enum Class:

Defines user types.

- ROLE_DOCTOR
- ROLE_ADMIN

3.1.2 User Class:

Holds information about the user.

User Class
Attributes
private string ID

private string name
private string userName
private string password
private List<DataSet> DataSets
private List<AuditRecord> auditRecords
Methods
Getters
Setters

3.1.3 DATA_TYPE Enum Class:

Defines type of data.

- Attributes:
    - DATA_TYPE_ULTRASOUND

3.1.4 DataPacket Class:

Defines and contains data that will be used system wide. It is raw data that is not preprocessed.

DataPacket Class
Attributes
private string ID
private DATA_TYPE type
private string name
private string filePath
private DateTime dateTime
private List<DataSet> dataSets
Methods
Getters
Setters

3.1.5 SETTING_TYPE Enum Class:

Defines type of a setting. Ex: INTEGER for filter kernel size

- Attributes:
    - SETTING_TYPE_INTEGER
    - SETTING_TYPE_FLOAT
    - SETTING_TYPE_STRING

3.1.6 PreproccesingSetting<T> Interface Class:

Defines a setting for preprocessing operation

PreprocessingSetting
Attributes
private T value
Methods
public T Getters & Setters
public SETTING_TYPE getType()
public static <T> PreprocessingSetting<T> createSetting(SETTING_TYPE type)

3.1.7 PreprocessingOperation Interface Class:

Defines how preprocessing class should be

PreprocessingOperation Interface Class
Attributes
private DATA_TYPE dataType
private List<PreprocessingSetting> settings
Methods
public virtual DataSet applyOp(DataPacket inputData)
Getters
Setters

3.1.8 AiResult Class:

Contains information about AI diagnosis results. Ex: name=Confidence, value = 0.843

AiResult Class
Attributes
private string ID
private List<string> values

private Dataset dataset
private List<Report> reports
Methods
Getters
Setters

3.1.9 Dataset Class:

Contains AI input data with applied preprocessing operations with AI results

Dataset Class
Attributes
private string ID
private string name
private string filePath
private List<PreprocessingOperation> operations
private DateTime dateTime
private User user
private DataPacket dataPacket
private AiResult result
Methods
public getters & setters

3.1.10 Preprocessor Class:

Contains available preprocessing options systemwide. Responsible for applying operations to the data

Preprocessor Class
Attributes
private Set<PreproccesingOperation> operations
private DATA_TYPE dataType
Methods
public bool addOperation(PreproccesingOperation op)
public bool removeOperation(PreproccesingOperation op)

Getters
Setters

3.1.11 AiClient Class:

Responsible for sending preprocessed data to AI modules

AiClient Class
Attributes
private string ip
private int port
private string name
private string description
private DATA_TYPE dataType
Methods
Getters
Setters

3.1.12 AUDIT_ACTION_TYPE Enum Class

Defines actions that will logged

- Attributes:
  - AUDIT_ACTION_TYPE_CHANGE_PASSWORD
  - AUDIT_ACTION_TYPE_CHANGE_ROLE
  - AUDIT_ACTION_TYPE_CHANGE_PREPROCCESS
  - AUDIT_ACTION_TYPE_APPLY_PREPROCESS

3.1.13 AuditRecord Class

Contains information about audit records like who did what or/and on whom.

AuditRecord Class
Attributes
private string ID
private AUDIT_ACTION_TYPE actionType
private string description
private datetime timestamp

private User user
Methods
Getters
Setters

3.1.14 Report Class

Contains information about generated report documents of AI analysis.

Report Class
Attributes
private string ID
private string fileType
private string fileName
private string filePath
private string userComment
private List<AiResult> aiResult
Methods
Getters
Setters

3.2 Repository

Each repository shall be implemented as an interface and extend JpaRepository with the type of the class and Long (ex. public interface ObjectRepository extends JpaRepository<ObjectModel ,Long>) JpaRepository will give abilities to access the database with its default implementation.

3.2.1 DataPacketRepository extends JpaRepository<DataPacket, Long>:

DataPacketRepository
Methods
Basic CRUD methods

3.2.2 UserRepository extends JpaRepository<User, Long>:

UserRepository
Methods
Basic CRUD methods

3.2.3 DatasetPacketRepository extends JpaRepository<Dataset, Long>:

DatasetPacketRepository
Methods
Basic CRUD methods

3.2.4 AuditRecordRepository extends JpaRepository<AuditRecord, Long>:

AuditRecordRepository
Methods
Basic CRUD methods

3.2.5 ReportRepository extends JpaRepository<AuditRecord, Long>:

ReportRepository
Methods
Basic CRUD methods

3.3 Service

3.3.1 UserService Class

Responsible for user login

UserService
Attributes
private UserRepository userRepository
Methods
public bool login(string name, string pw)

3.3.2 PreprocessService Class

Responsible for preprocess operations. Holds information about operations on data types

PreprocessService
Attributes
private Set<Preprocessor> preprocessors
Methods
public List<PreprocessOperation> getOperations(DATA_TYPE type)
public DataSet applyOperations(DataPacket dataPacket, List<PreprocessOperation> oL)
Getters and setters

3.3.3 AiModuleService Class

Responsible for getting results from Ai modules and create Ai results

AiModuleService Class
Attributes
private Set<AiClients> modules
Methods
public AiResult getResults(DataSet pD, string moduleName)
Getter and setter

3.3.4 DataSetService Class

Responsible for creating datasets

DataSetService Class
Attributes
private DataSetRepository datasetRepository
Methods
public DataSet createDataset(DataPacket cD, User user, List<PreprocessOperation> oD)

3.3.5 AuditService Class

Responsible for creating records of user specific actions

AuditService
Attributes
private AuditServiceRepository auditServiceRepository
Methods
public AuditRecord createAuditRecord(User us, AUDIT_ACTION_TYPE aT, string des)

3.3.6 ReportService Class

Responsible for creating reports of AI results

ReportService
Attributes
private ReportServiceRepository reportServiceRepository
Methods
public Report createReport(AiResult aiResult, string fileType, string fileName, string filePath, string userComment)

3.4 Controller

All Controller endpoints enforce a strict Service-to-ResponseEntity pattern. Each response is wrapped in a ResponseEntity that carries a system-generated watermark. This watermark serves as a unique fingerprint for the operation, linking the result to the corresponding entry in the Audit Trail (FR10) for regulatory compliance.

3.4.1 UserController Class , endpoint: ‘api/user’

Responsible for user login

UserController Class
Attributes
private UserService userService
Methods
public ResponseEntity<Bool> login(string Username, string password)
public ResponseEntity<Bool> logout()

3.4.2 PreprocessController Class endpoint: ‘api/preprocess’

Responsible for preprocess operations. Holds information about operations on data types

PreprocessController Class

Attributes
private PreprocessorService preprocessorService
Methods
public ResponseEntity<List<PreprocessOperation> >getOperations(DATA_TYPE type)
public ResponseEntity<DataSet> applyOperations(DataPacket rawData, User user, List<PreprocessOperation> operationsInOrder)
Getters and setters

3.4.3 AiModuleController Class endpoint: ‘api/aimodule’

Responsible for getting results from Ai modules and create Ai results

AiModuleController Class
Attributes
private AiModuleService aimoduleService
Methods
public AiResult getResults(DataSet preprocessedData)
Getters and setters

3.4.4 DataSetController Class endpoint: ‘api/dataset’

Responsible for creating datasets

DataSetController Class
Attributes
private DatasetService datasetService
Methods
public ResponseEntity<DataSet> createDataset(DataPacket rawData, User user, List<PreprocessOperation> operationsInOrder)

3.4.5 AuditController Class endpoint: ‘api/audit’

Responsible for creating records of user specific actions

AuditController Class
Attributes
private AuditService auditService
Methods

public ResponseEntity<AuditRecord> createAuditRecord(User user, AUDIT_ACTION_TYPE actionType, string description)

3.4.6 ReportController Class endpoint: 'api/report'

Responsible for creating report documents of Ai results

ReportController Class
Attributes
private ReportService reportService
Methods
public ResponseEntity<Report> createReport(AiResult,string filename, string filepath, string fileType, string userComment)

3.5 View

3.5.1 LoginView

Displays login screen. User must login to use application

LoginView
Attributes
private string username
private string password
Methods
private void login(string username, string password)
private void displayError()

3.5.2 MainView

Displays main screen. User must login to use application. In this view user can choose a datatype that he wants to work on. Admin panel is for admin users only.

MainView
Methods
private void selectDataType(DataType type)
private void goToAdminPanel()
private void logOut()

3.5.3 DataSelectionView

This class allows the user to select the data they will work with

DataSelectionView
Attributes
private DataType type
private List<DataPacket> requestedData
private List<DataPacket> selectedData
Methods
private void queryById(string id)
Getter and setters

3.5.4 PreprocessSettingView

This class displays preprocessing options and let users change them .

Preprocessing View
Attributes
private List<PreprocessingOperation> suitableOperations
private List<PreprocessingOperation> selectedOperations
Methods
private void changeSetting(PreprocessingOperation op, string name, SETTING_TYPE value)
Getter and Setters

3.5.5 AiSelectionView

This class displays ai module options to user.

AiSelectionView
Attributes
private List<AiClient> Clients
private AiClient selectedClient
private List<DataSet> datasets
Methods
private void startAnalysis()
private void displayResults()
Getter and Setters

3.5.6 ReportView

This class displays ai reports to users. User have an option to add comment, export report and share it

ReportView
Attributes
private List<Report> reports
private string comment
Methods
private Report export(string fileType)
private void addComment(string comment)
Getter and Setters

3.5.7 AdminPanelView

This class displays user information and let user change some settings.

AdminPanelView
private List<User> users
Methods
private changePassword(User user,string password)
private bool createUser(string name, string userName, string password, string role)
private bool deleteUser (User user)
public bool updateName (User user, string newName)
public bool updateUserName(User user, string newUserName)

4. Glossary

AI Module: External artificial intelligence service providing diagnostic results.

Dataset: A processed ultrasound instance with preprocessing history and AI result.

ts.**Audit Trail:** Persistent log of system actions.

ROI: Region of Interest selected during cropping.

Watermark: Unique transaction identifier attached to every API response.

Factory Design Pattern: A creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created based on a specific type or enum.

JPA (Java Persistence API): A Java specification used for managing relational data in applications using Java Platform, Standard Edition, and Java Platform, Enterprise Edition.

JSON (JavaScript Object Notation): A lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.

REST (Representational State Transfer): An architectural style for providing standards between computer systems on the web, making it easier for systems to communicate with each other via HTTP.

ResponseEntity: A Spring Framework class that represents the entire HTTP response, including the status code, headers, and the body of the response.

5. References

- https://bilkent-musync.github.io/docs/Musync%20-%20Low-Level%20Design%20Report.pdf
- https://docs.spring.io/spring-boot/index.html
- https://www.w3schools.com/java/java_oop.asp
- https://www.geeksforgeeks.org/system-design/software-design-patterns/
- https://react.dev/reference/react